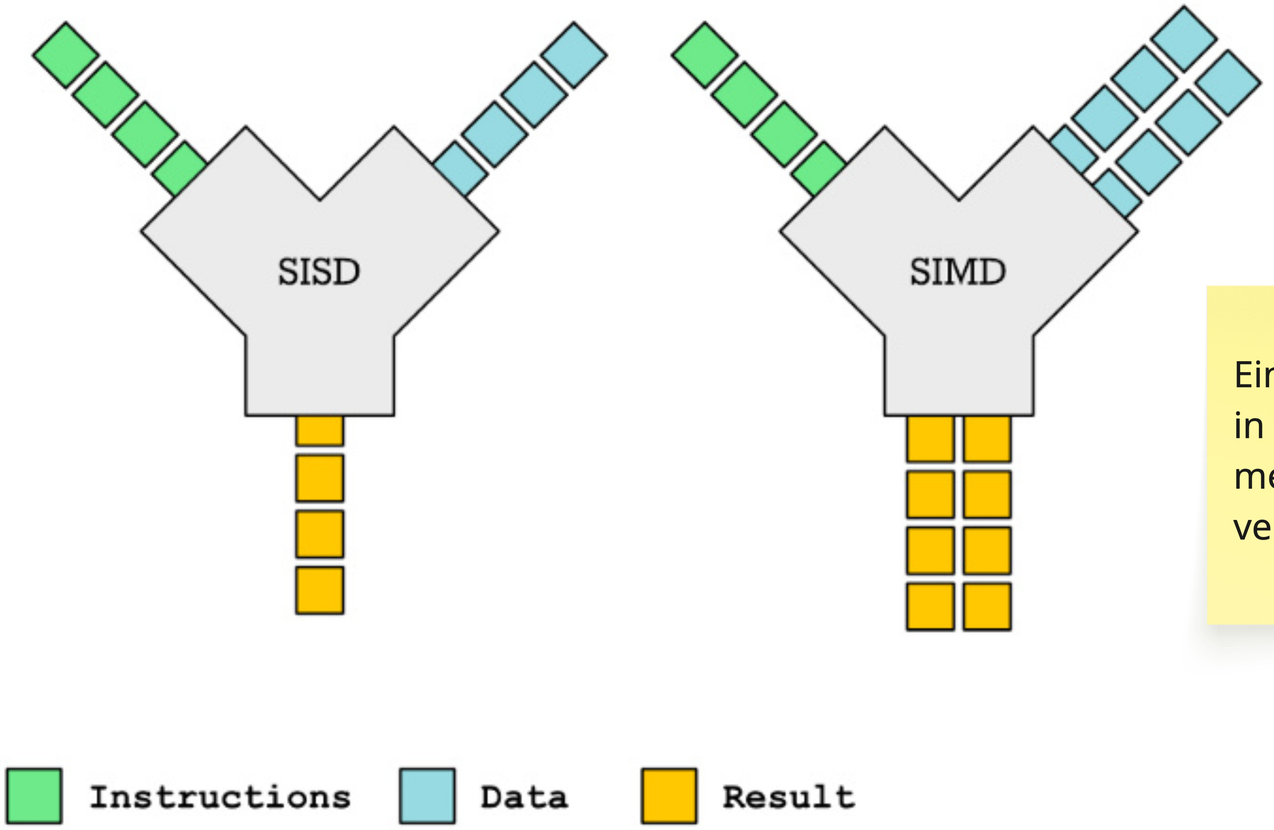
Vector API



Ein **SIMD** kann in einem Schritt mehrere Daten verarbeiten

Vector API

- **Vector API** = expliziter, portabler Zugriff auf **SIMD**-Instruktionen (Datenparallelismus).
- Ziel: nahe an manuellem Code zur Vektorisierung, aber mit **HotSpot**-Optimierungen und **sicherem Java-Code**.
- Typische Gewinne: **2–10×** bei großen, speicherlokalen, elementweisen Rechenlasten (z.B. DSP, Bildverarbeitung).

Vector API - Konzepte

- **Species** (VectorSpecies<T>): bestimmt Vektorbreite (z. B. 256-bit) & Elementtyp.
- **Vector**: die eigentlichen Werte
- **Mask**: selektive Aktivierung einzelner Lanes (Tail-Handling, Bedingungen).

Beispiel: Skalares addieren

```
public static void addScalar(float[] a, float[] b, float[] out) {
    int n = a.length;
    for (int i = 0; i < n; i++) {
        out[i] = a[i] + b[i];
    }
}
```

Species bestimmt, wie viele Werte die CPU *gleichzeitig* verarbeiten kann

SPECIES_PREFERRED überlässt Java die Entscheidung, was am besten passt

```java
import jdk.incubator.vector.FloatVector;
import jdk.incubator.vector.VectorMask;
import jdk.incubator.vector.VectorSpecies;

public class VecAdd {
    private static final VectorSpecies<Float> SPECIES = FloatVector.SPECIES_PREFERRED;

    public static void addVector(float[] a, float[] b, float[] out) {
        int i = 0, n = a.length;
        int upperBound = SPECIES.loopBound(n);
        // Hauptschleife: volle Vektorbreite
        for (; i < upperBound; i += SPECIES.length()) {
            var va = FloatVector.fromArray(SPECIES, a, i);
            var vb = FloatVector.fromArray(SPECIES, b, i);
            var vc = va.add(vb);
            vc.intoArray(out, i);
        }
        // Tail: Rest-Elemente per Masken
        if (i < n) {
            VectorMask<Float> m = SPECIES.indexInRange(i, n);
            var va = FloatVector.fromArray(SPECIES, a, i, m);
            var vb = FloatVector.fromArray(SPECIES, b, i, m);
            var vc = va.add(vb, m);
            vc.intoArray(out, i, m);
        }
    }
}
```

```
Iteration 1: nimmt a[0–7] + b[0–7]
Iteration 2: nimmt a[8–15] + b[8–15]
usw.
```

Wenn die Arraylänge **kein Vielfaches von 8** ist (z. B. 10 Elemente), dann bleiben am Ende 2 übrig.

Mask (m) sorgt dafür, dass nur die verbleibenden 2 Positionen verarbeitet werden, ohne über das Array-Ende hinauszuschreiben.

Foreign memory API

Foreign Memory API

- Die Foreign Function & Memory API (FFM) ist seit JDK 22 produktiv
- Kann off-heap-Speicher sicher adressieren
- mit nativen Bibliotheken zu interagieren – ohne JNI.

MemorySegment und Arena

```
import java.lang.foreign.*;
import static java.lang.foreign.ValueLayout.*;

void main() {
    try (Arena arena = Arena.ofConfined()) {
        // 10 ints off-heap, alignment gemäß Layout
        MemorySegment ints = arena.allocate(JAVA_INT, 10);
        for (long i = 0; i < 10; i++) {
            ints.setAtIndex(JAVA_INT, i, (int) (i * 10));
        }
        // Lesen per Convenience-API
        int v = ints.getAtIndex(JAVA_INT, 3); // 30
    }
}
```

Arena

- Arena.global() – unbounded, nie freigegeben
- Arena.ofAuto() – GC-verwaltet, bounded
- Arena.ofConfined() – manuell schließbar, thread-konfiniert
- Arena.ofShared() – manuell schließbar, thread-geteilt

```
import java.lang.foreign.*;
import static java.lang.foreign.ValueLayout.*;

void main() {
    MemorySegment s;
    try (Arena arena = Arena.ofConfined()) {
        s = arena.allocate(100); // 100 Bytes
    } // -> Deallokation/Invalidierung
    // s.get(JAVA_BYTE, 0); // IllegalStateException
}
```

Arena.ofConfined()
wird geschlossen,
wenn der Block endet.

Layouts

Layouts beschreiben die Struktur von Speicher (Größe, Alignment, Endianness)

```
import java.lang.foreign.*;
import java.lang.invoke.VarHandle;
import static java.lang.foreign.ValueLayout.*;
import static java.lang.foreign.MemoryLayout.PathElement.*;

void main() {
    StructLayout POINT = MemoryLayout.structLayout(
            JAVA_INT.withName("x"),
            JAVA_INT.withName("y")
    );
    VarHandle VH_X = POINT.varHandle(groupElement("x"));
    VarHandle VH_Y = POINT.varHandle(groupElement("y"));

    try (Arena arena = Arena.ofConfined()) {
        // Array (Sequence) von 3 Points
        MemorySegment points = arena.allocate(POINT, 3);

        // Point[1] setzen
        long idx1 = 1L;
        long elemSize = POINT.byteSize();
        MemorySegment p1 = points.asSlice(idx1 * elemSize, elemSize);
        VH_X.set(p1, 10);
        VH_Y.set(p1, 20);
    }
}
```

FileChannel

- Seit JDK 22 bietet FileChannel.map(MapMode, offset, size, Arena) direkt einen MemorySegment statt nur MappedByteBuffer.
- Ideal für Zero-Copy-Zugriffe (z.B. große Logs, Binärformate).

```
import java.lang.foreign.*;
import java.nio.file.*;
import java.nio.channels.FileChannel;
import java.nio.file.StandardOpenOption.*;
import static java.lang.foreign.ValueLayout.*;


void main() {
    Path path = Path.of("access.log");
    try (FileChannel ch = FileChannel.open(path, java.util.Set.of(READ));
        Arena arena = Arena.ofShared()) {

        MemorySegment m = ch.map(FileChannel.MapMode.READ_ONLY, 0, ch.size(), arena);

        long lines = 0;
        for (long i = 0, n = m.byteSize(); i < n; i++) {
            if (m.get(JAVA_BYTE, i) == (byte) '\n') lines++;
        }
        System.out.println("Lines: " + lines);
    }
}
```

Downcalls

```
import java.lang.foreign.*;
import java.lang.invoke.MethodHandle;
import static java.lang.foreign.ValueLayout.*;
void main() {

    Linker linker = Linker.nativeLinker();
    MethodHandle strlen = linker.downcallHandle(
            linker.defaultLookup().findOrThrow("strlen"),
            FunctionDescriptor.of(JAVA_LONG, ADDRESS) // size_t strlen(const char*)
    );

    try (Arena arena = Arena.ofConfined()) {
        MemorySegment cStr = arena.allocateFrom("Panama FFM");
        long len = (long) strlen.invokeExact(cStr);
        System.out.println(len); // 11
    }
}
```

Hierfür ist kein JNI nötig